HW4.5 GPU Assignment II: Training Throughput and Memory Engineering

Repository: `shulamite512/hw4.5_GPU`
Measured hardware: NVIDIA GeForce RTX 4090, 23.99 GiB, host ADS-R15-836-04
Dataset: `cardiffnlp/tweet_eval` sentiment, concatenated splits
SLICE: 384 | **SEED:** 1384 | **Working rows:** 384-5383 inclusive (5,000 rows)

Executive summary

The repository contains a reproducible HW4.a and HW4.5 measurement set for one RTX 4090. The BERT baseline reached 0.6907 test accuracy using the best-validation-loss checkpoint, while mixed-precision training achieved the best measured throughput at 767.0 samples/s. The largest valid baseline batch was 512, with 1,145.3 samples/s and 20.591 GiB allocated memory; batch 1024 was rejected after enforcing a 99% VRAM cap. vLLM was substantially faster than the Hugging Face pipeline, reaching 5,546.5 tokens/s at batch 32 in the HW4.5 rerun.

1. HW4.a BERT sentiment classifier

Data and training setup

The 5,000-row working subset was split stratified into 3,500 training, 750 validation, and 750 test rows. The classifier used `bert-base-uncased`, maximum sequence length 64, batch size 32, four epochs, learning rate 2e-5, fp32, and AdamW. The three classes were negative, neutral, and positive.

Epoch	Train loss	Validation loss	Validation accuracy	Train time (s)	Samples/s
1	0.8913	0.7855	0.6213	5.29	661.5
2	0.6255	**0.7108**	0.6827	4.92	711.1
3	0.4663	0.7519	0.6853	5.04	695.0
4	0.3503	0.7814	**0.6867**	5.02	697.4

Validation loss stopped improving after epoch 2. Training continued for two additional epochs, costing 10.06 seconds; the final-epoch test accuracy was 0.6880 versus 0.6907 for the best-validation-loss checkpoint.

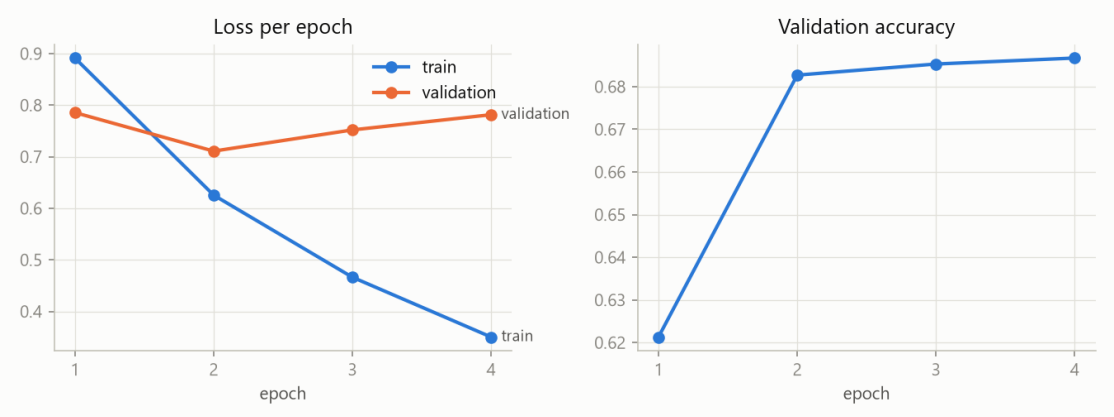


Figure 1: HW4.a training curves

Test performance

Test accuracy after saving and reloading the checkpoint was **0.6907**.

True \ Pred	Negative	Neutral	Positive
Negative	73	35	9
Neutral	35	238	69
Positive	11	73	207

Class	Precision	Recall	F1	Support
Negative	0.6134	0.6239	0.6186	117
Neutral	0.6879	0.6959	0.6919	342
Positive	0.7263	0.7113	0.7188	291

The worst class was **negative**, with the lowest F1. Five negative examples were misclassified:

1. `@user even I didn't know it's 18th sept at KL Live to be exact :p` -> neutral. There are few negative words and the playful emoticon supports a neutral interpretation; the gold label may contain noise.
2. `Assam Congress leader Nilomoni sen deka reffered Smritini Irani as PM's 2nd wife..And Still Rahul Gandhi talks about women Empowerment.` -> neutral. The negative meaning is ironic and surrounded by factual, news-like wording.
3. `Don't want to hear Seth Rollins Speech about how great he is tomorrow bruh.` -> positive. The model likely focused on "how great he is" and missed the negation scope.
4. `Not happy that I'm working through the advance screening tomorrow, so still have to wait OVER TWO WEEKS to see Paper Towns #wallofcow` -> neutral. Positive context words compete with the initial "Not happy" signal.
5. `@user Hi, was offered the 6 months free Netflix last Tuesday and was told it takes 5 days to receive a code but haven't got anything` -> neutral. The complaint is expressed as a polite factual timeline rather than explicit sentiment.

2. HW4.a serving benchmark

Model: `Qwen/Qwen2.5-1.5B-Instruct`, bf16, 128 generated tokens per request.

Configuration	Tokens/s	p50 latency (ms)	p95 latency (ms)
Hugging Face pipeline, batch 1	62.9	2,025.1	2,112.6
vLLM, batch 1	203.8	626.1	646.3
vLLM, batch 8	1,380.8	738.3	771.0
vLLM, batch 32	5,546.5	739.1	740.1

Relative to the Hugging Face baseline, vLLM latency remained lower at every tested batch size while throughput increased from 203.8 to 5,546.5 tokens/s. Relative to vLLM batch 1, latency rose at batch 8 and batch 32 while throughput continued improving, so batch 8 is the first practical latency-throughput trade-off point. For interactive applications, p95 per-request latency matters more because it determines how long a user waits; total throughput matters more for batch serving and cost efficiency.

3. Part A: batch scaling

Batch	Throughput (samples/s)	Peak allocated (GiB)	Peak reserved (GiB)	Time/epoch (s)
1	28.9	2.061	2.279	121.10
2	63.9	2.062	2.279	54.81
4	121.3	2.067	2.336	28.87
8	226.7	2.066	2.270	15.44
16	477.5	2.088	2.264	7.33
32	708.2	2.472	2.770	4.94

Batch	Throughput (samples/s)	Peak allocated (GiB)	Peak reserved (GiB)	Time/epoch (s)
64	920.8	3.726	4.012	3.80
128	1,022.7	6.110	6.275	3.42
256	1,086.3	10.978	11.125	3.22
512	**1,145.3**	20.591	20.766	3.06

Batch 512 was the largest valid run; batch 1024 produced OOM after the 99% VRAM cap was applied. Throughput reached 95% of its observed maximum at batch 256 and improved only modestly thereafter while memory continued to grow rapidly. Only one card was measured, so the required two-card comparison and maximum-batch ratio cannot be reported.

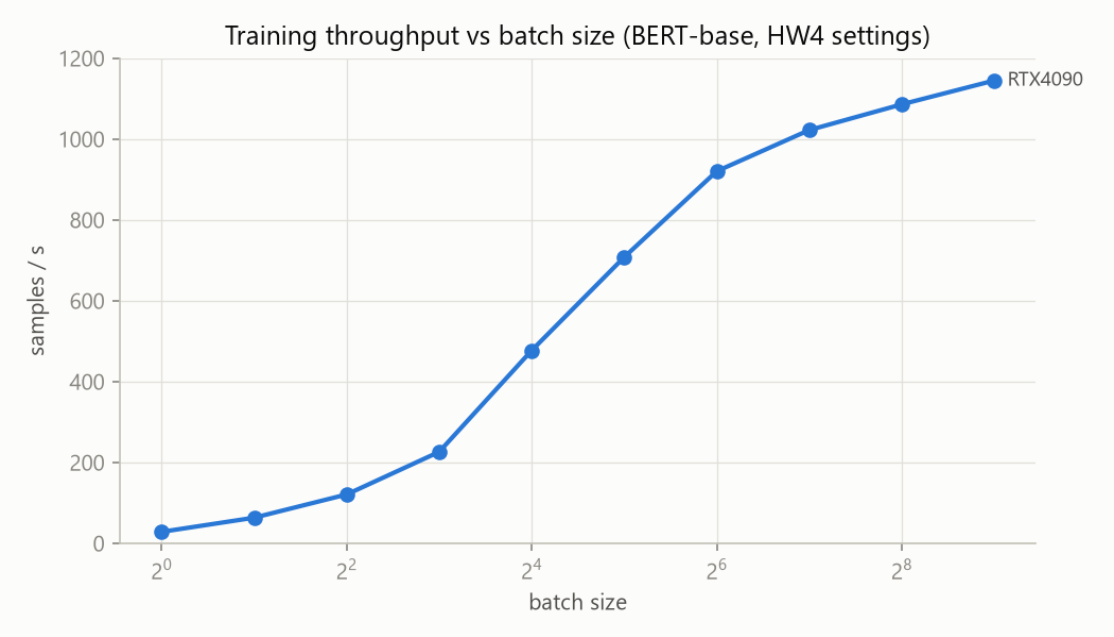


Figure 2: Part A throughput versus batch size

4. Part B: memory techniques

Measurements below use batch 32 for comparable work; max batch is the largest tested power of two that ran.

Technique	Max batch	Throughput (samples/s)	Peak memory (GiB)	Eval accuracy
Baseline	512	699.6	2.472	0.6880
Mixed precision (bf16)	512	**767.0**	2.256	0.6893
Gradient checkpointing	2,048	559.5	2.070	0.6880
Gradient accumulation (8 x 16)	128 effective	432.3	2.066	0.6747
8-bit AdamW	512	668.8	1.870	0.6840
Best combo: AMP + 8-bit AdamW	512	703.4	1.635	0.6867

Technique	Max batch	Throughput (samples/s)	Peak memory (GiB)	Eval accuracy
AMP + 8-bit AdamW + checkpointing	4,096	550.6	**1.082**	0.6867

Mixed precision is the first technique to apply: it saved memory, improved throughput, and preserved accuracy. Gradient checkpointing is the least attractive technique at batch 32 because its 20.0% throughput loss buys only 0.402 GiB. On a 32 GB card, checkpointing would not be necessary for the measured HW4 workload, although it becomes useful for much larger batches.

Gradient accumulation reached effective batch 128 using physical batch 16 and accumulation factor 8. It took 5.922 seconds for 20 optimizer steps versus 2.507 seconds for direct batch 128, and its lower accuracy is consistent with fewer optimizer steps per epoch under the unchanged learning rate.

5. Part C: distributed training

The repository contains only a single-GPU DDP baseline on one RTX 4090. The run was homogeneous, single-node, world size 1, and had no all-reduce communication. The reported communication column is therefore a timing-noise floor rather than a two-GPU communication measurement.

Batch/GPU	Throughput	Step (ms)	Compute (ms)	Reported comm (ms)
8	163.0 samples/s	49.09	54.59	0.00
16	296.2 samples/s	54.02	46.51	7.51
32	588.2 samples/s	54.40	50.41	4.00
64	932.2 samples/s	68.65	67.34	1.31
128	1,036.0 samples/s	123.55	123.37	0.18
256	1,094.0 samples/s	234.00	236.48	0.00

Two-GPU throughput, scaling efficiency, interconnect path, effective bandwidth, and the batch size where two GPUs become worthwhile remain unmeasured. The repository records that the TA confirmed the two-GPU pairing was optional on 2026-10-22.

6. Part D: serving and KV cache

The HW4.5 serving rerun used the same RTX 4090, bf16 precision, model, and generation length as HW4.a. The results agree within approximately 2%, supporting the conclusion that there was no material hardware, precision, or measurement change.

At batch 8, HF Transformers measured 28 KiB of KV cache per token per sequence. The measured HF memory limit was 58,368 tokens per sequence; vLLM's pool allowed 84,680 tokens per sequence, but the model's configured maximum position length was 32,768. Therefore the longest model-supported context was 32,768 tokens on this card.

Detailed KV-cache measurements

Context length	HF KV measured (GiB)	HF analytic KV (GiB)	HF peak total (GiB)
1,024	0.227	0.219	3.607
2,048	0.445	0.438	3.828
3,072	0.664	0.656	4.049
4,096	0.883	0.875	4.268
8,192	1.758	1.750	5.260
16,384	3.508	3.500	7.518
32,768	7.008	7.000	12.033
58,368	12.477	12.469	19.089

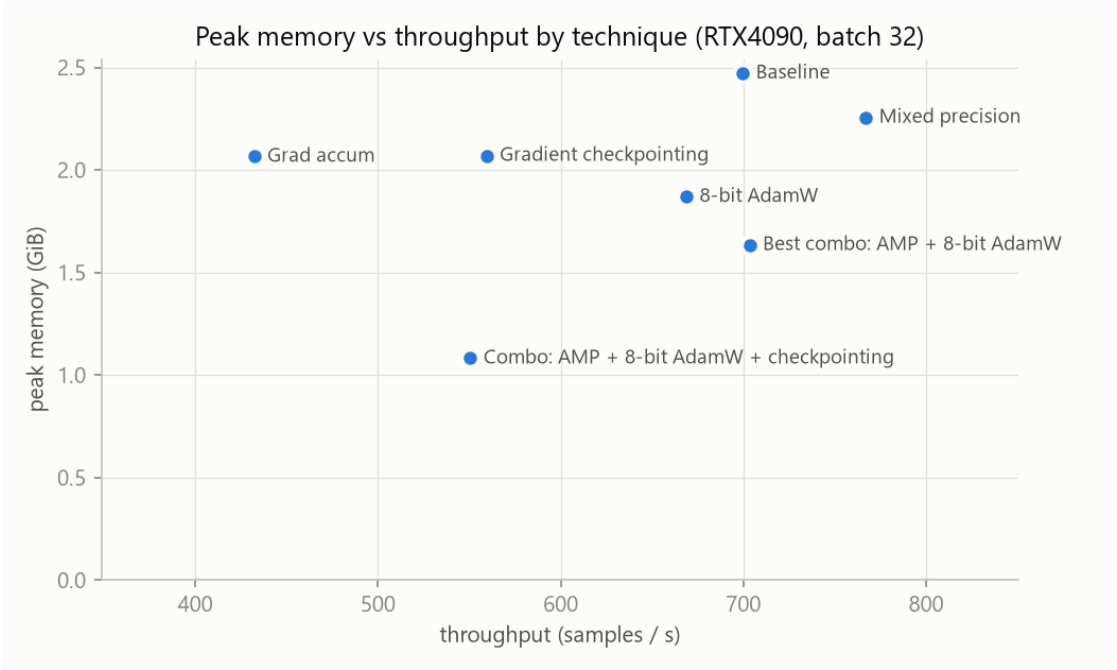


Figure 3: Part B memory versus throughput

The measured and analytic values differ by at most about 8 MiB at the tested points. The linear growth is expected because the KV cache stores keys and values for every token in every sequence. For vLLM, the configured pool contained 677,440 tokens, equivalent to 18.09 GiB at 90% GPU utilization. At batch 8 this corresponds to 84,680 tokens per sequence by memory capacity, but the model configuration limits the usable context to 32,768 tokens.

7. Experimental methodology and implementation details

Data preparation

The scripts concatenate the TweetEval sentiment train, validation, and test splits so that the assignment's consecutive-row requirement is explicit. The selected slice begins at row 384, derived from SID4 = 1384, and ends at row 5383. No rows were dropped for missing text. Labels are encoded as negative, neutral, and positive, and the 5,000 selected rows are split with stratification into 3,500 training, 750 validation, and 750 test examples.

Training implementation

`hw4a_train.py` tokenizes tweets with `BertTokenizer`, pads/truncates to 64 tokens, constructs tensor datasets, initializes `BertForSequenceClassification`, trains with AdamW, evaluates after every epoch, saves the best validation checkpoint, reloads it, and runs custom inference. The seed 1384 is applied to the data split, model initialization, and training setup. The memory experiments reuse the same model and data recipe while changing one memory technique at a time.

Measurement definitions

Peak allocated memory is `torch.cuda.max_memory_allocated()` and peak reserved memory is also recorded to show allocator-cache behavior. Part A and Part B throughput are measured over 20 timed optimizer steps after warm-up. Time per epoch is computed from the training split size divided by measured samples per second. Serving throughput counts generated tokens only; every serving request generates 128 tokens, and request latency is the wall time of the batch containing that request.

Software and hardware provenance

Item	Value
Host	ADS-R15-836-04, Windows 11 with WSL2 for vLLM/DDP
GPU	NVIDIA GeForce RTX 4090, 23.99 GiB
GPU UUID	GPU-c5ad1a63-b7c0-1838-5d8f-0df577ec82ce
Driver	610.60
CUDA	12.8
Training Python	3.13.14
Training PyTorch	2.11.0+cu128
Serving Python	3.12.3
Serving PyTorch	2.8.0+cu128
Transformers	5.20.2 for training; 4.57.6 for serving/vLLM compatibility
vLLM	0.11.0
Model	`bert-base-uncased` for classification; `Qwen/Qwen2.5-1.5B-Instruct` for serving

The Windows driver initially allowed large batches to spill into system memory instead of raising a true GPU OOM. The experiment scripts therefore enforce a 99% per-process VRAM cap. The invalid spill runs are retained in the run log but are excluded from the reported Part A measurements.

8. GPU-hour and run provenance

The generated GPU-hour record totals 1.64 GPU-hours on the RTX 4090. It includes smoke tests, failed and interrupted runs, valid training experiments, serving benchmarks, KV-cache measurements, and DDP baseline runs. Including unsuccessful attempts is important because they consumed reserved GPU time.

Activity	Status	Approx. GPU-hours
Setup and smoke tests	completed/failed	0.076
HW4.a training	completed	0.025
Invalid Part A attempts and cap verification	interrupted/completed	0.616
Valid Part A	completed	0.067
Part B memory techniques	completed	0.331
HW4.a serving benchmarks	completed/failed	0.184
Part D KV-cache benchmarks	interrupted/completed	0.279
Part C single-GPU DDP baselines	completed	0.029
Total		**1.64**

9. Requirement-to-evidence checklist

Assignment requirement	Evidence in repository	Status
BERT preprocessing, training, evaluation, save/reload	`hw4a_train.py`, `results/hw4a_train_RTX4090.json`, `REPORT.md`	Complete
Exact slice, split, and seed	`results/hw4a_train_RTX4090.json` and report	Complete
Epoch losses, validation accuracy, stopping analysis	`REPORT.md` and training JSON	Complete
Confusion matrix, class metrics, five examples	`REPORT.md`, `diagnoses.json`	Complete
GPU memory and wall-clock baseline	`METRICS.md`, training JSON	Complete
HF and vLLM serving comparison	serving JSON files and report table	Complete for RTX 4090

Assignment requirement	Evidence in repository	Status
Batch scaling and OOM boundary	`results/partA_RTX4090.json`, Part A plot	Complete for RTX 4090
Memory techniques and best combination	`results/partB_RTX4090.json`, Part B plot	Complete for RTX 4090
Two-GPU DDP scaling	`results/partC_w1.json`	Single-GPU baseline only; second GPU optional/not run
KV cache versus context length	Part D JSON files and detailed table above	Complete for RTX 4090
GPU-hour record	`GPU_HOURS.md`, `RUN_LOG.txt`	Complete
UUID capture	`captures/uuid_*.txt`	One used-GPU capture present
Reservation confirmation	`RESERVATION.md` and reservation attachment	Reservation details still need completion
`hw4-5` repository tag	Git tag	Created/prepared; push with final upload

10. Part E: final configuration and GPU hours

For a full fine-tune on the measured card, the recommended configuration is bf16 mixed precision at the original batch size 32: it was the fastest measured configuration with accuracy (767.0 samples/s, 0.6893 accuracy), reduced memory relative to baseline, and preserved the original optimization recipe. Using 59,899 rows, four epochs, and the measured throughput gives 167,716 training samples / 767.0 samples/s = 219 seconds, or approximately **0.06 GPU-hours** excluding validation passes.

The repository's generated GPU-hour record totals **1.64 GPU-hours** for the RTX 4090, including failed, interrupted, smoke-test, and valid runs.

11. Reproducibility references

The numeric results in this report are drawn from the repository's `results/*.json`, `outputs/*.log`, `RUN_LOG.txt`, `METRICS.md`, and `REPORT.md`. The repository's scripts can regenerate the report tables and plots with `python make_report.py` after any additional card results are copied into `results/`.